PAYTM DSA INTERVIEW

Data Structures & Algorithms

Most Asked Questions | Senior Backend Engineer | 2025

20+	**5**	**Arrays**	**O(n)**

SECTION 1 · ARRAYS & STRINGS

Q1. Two Sum — Find indices of two numbers that add up to target — Easy

■ Array · HashMap · O(n)

```
public int[] twoSum(int[] nums, int target) {
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
int complement = target - nums[i];
if (map.containsKey(complement))
return new int[]{map.get(complement), i};
map.put(nums[i], i);
}
throw new IllegalArgumentException("No solution");
}
```

■ **Time: O(n) | ■ Space: O(n)**

Q2. Longest Substring Without Repeating Characters — Medium

■ Sliding Window · HashMap · String

```
public int lengthOfLongestSubstring(String s) {
Map<Character, Integer> map = new HashMap<>();
int maxLen = 0, left = 0;
for (int right = 0; right < s.length(); right++) {
char c = s.charAt(right);
if (map.containsKey(c))
left = Math.max(left, map.get(c) + 1);
map.put(c, right);
maxLen = Math.max(maxLen, right - left + 1);
}
return maxLen;
}
// "abcabcbb" → 3 | "pwwkew" → 3 | "bbbbb" → 1
```

■ **Time: O(n) | ■ Space: O(min(m,n))**

Q3. Maximum Subarray Sum (Kadane's Algorithm) — Medium

■ Array · DP · Greedy

```
public int maxSubArray(int[] nums) {
int maxSum = nums[0], currentSum = nums[0];
for (int i = 1; i < nums.length; i++) {
currentSum = Math.max(nums[i], currentSum + nums[i]);
maxSum = Math.max(maxSum, currentSum);
```

```
}
return maxSum;
}
// [-2,1,-3,4,-1,2,1,-5,4] → 6 (subarray [4,-1,2,1])
```

■ Time: O(n) | ■ Space: O(1)

Q4. Trapping Rain Water — Hard

■ Array · Two Pointers · Stack

```
public int trap(int[] height) {
int left = 0, right = height.length - 1;
int leftMax = 0, rightMax = 0, water = 0;
while (left < right) {
if (height[left] < height[right]) {
leftMax = Math.max(leftMax, height[left]);
water += leftMax - height[left];
left++;
} else {
rightMax = Math.max(rightMax, height[right]);
water += rightMax - height[right];
right--;
}
}
return water;
}
// [0,1,0,2,1,0,1,3,2,1,2,1] → 6
```

■ Time: O(n) | ■ Space: O(1)

Q5. Best Time to Buy and Sell Stock (max profit) — Easy

■ Array · Greedy · One Pass

```
public int maxProfit(int[] prices) {
int minPrice = Integer.MAX_VALUE, maxProfit = 0;
for (int price : prices) {
minPrice = Math.min(minPrice, price);
maxProfit = Math.max(maxProfit, price - minPrice);
}
return maxProfit;
}
// [7,1,5,3,6,4] → 5 (buy at 1, sell at 6)
```

■ Time: O(n) | ■ Space: O(1)

SECTION 2 · LINKED LIST

Q6. Detect a Cycle in Linked List (Floyd's Algorithm) Medium

■ Linked List · Two Pointers · Cycle Detection

```
public boolean hasCycle(ListNode head) {
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next;
fast = fast.next.next;
if (slow == fast) return true;
}
return false;
}

// Bonus: Find cycle start node
public ListNode detectCycle(ListNode head) {
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next; fast = fast.next.next;
if (slow == fast) {
slow = head; // reset one pointer to head
while (slow != fast) { slow = slow.next; fast = fast.next; }
return slow; // cycle start
}
}
return null;
}
```

■ **Time: O(n) | ■ Space: O(1)**

Q7. Reverse a Linked List (Iterative + Recursive) Easy

■ Linked List · Recursion · Pointers

```
// Iterative
public ListNode reverse(ListNode head) {
ListNode prev = null, curr = head;
while (curr != null) {
ListNode next = curr.next;
curr.next = prev;
prev = curr;
curr = next;
}
return prev;
}
```

```
// Recursive
public ListNode reverseRecursive(ListNode head) {
if (head == null || head.next == null) return head;
ListNode newHead = reverseRecursive(head.next);
head.next.next = head;
head.next = null;
return newHead;
}
```

■ **Time: O(n) | ■ Space: O(1) iterative, O(n) recursive**

Q8. Merge Two Sorted Linked Lists — Easy

■ Linked List · Two Pointers · Merge

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
ListNode dummy = new ListNode(0), curr = dummy;
while (l1 != null && l2 != null) {
if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
else { curr.next = l2; l2 = l2.next; }
curr = curr.next;
}
curr.next = (l1 != null) ? l1 : l2;
return dummy.next;
}
```

■ **Time: O(m+n) | ■ Space: O(1)**

SECTION 3 · TREES & BINARY SEARCH

Q9. Lowest Common Ancestor (LCA) of a Binary Tree — Medium

■ Binary Tree · DFS · Recursion

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
if (root == null || root == p || root == q) return root;
TreeNode left = lowestCommonAncestor(root.left, p, q);
TreeNode right = lowestCommonAncestor(root.right, p, q);
if (left != null && right != null) return root; // p and q on different sides
return (left != null) ? left : right;
}
// Paytm context: find LCA in org-hierarchy or category-tree
```

■ **Time: O(n) | ■ Space: O(h) — h = tree height**

Q10. Validate Binary Search Tree — Medium

■ BST · DFS · Inorder Traversal

```
public boolean isValidBST(TreeNode root) {
return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

private boolean validate(TreeNode node, long min, long max) {
if (node == null) return true;
if (node.val <= min || node.val >= max) return false;
return validate(node.left, min, node.val)
&& validate(node.right, node.val, max);
}
// Use Long to handle Integer.MIN_VALUE and Integer.MAX_VALUE edge cases!
```

■ **Time: O(n) | ■ Space: O(h)**

Q11. Binary Tree Level Order Traversal (BFS) — Medium

■ BFS · Queue · Binary Tree

```
public List<List<Integer>> levelOrder(TreeNode root) {
List<List<Integer>> result = new ArrayList<>();
if (root == null) return result;
Queue<TreeNode> queue = new LinkedList<>();
queue.offer(root);
while (!queue.isEmpty()) {
int size = queue.size(); // snapshot current level size!
List<Integer> level = new ArrayList<>();
for (int i = 0; i < size; i++) {
TreeNode node = queue.poll();
```

```
    level.add(node.val);
    if (node.left != null) queue.offer(node.left);
    if (node.right != null) queue.offer(node.right);
    }
    result.add(level);
    }
    return result;
    }
```

■ Time: O(n) | ■ Space: O(w) — w = max width

SECTION 4 · DYNAMIC PROGRAMMING

Q12. Coin Change — Minimum coins to make amount — Medium

■ DP · BFS · Greedy won't work

```
public int coinChange(int[] coins, int amount) {
int[] dp = new int[amount + 1];
Arrays.fill(dp, amount + 1); // sentinel infinity
dp[0] = 0;
for (int i = 1; i <= amount; i++) {
for (int coin : coins) {
if (coin <= i)
dp[i] = Math.min(dp[i], dp[i - coin] + 1);
}
}
return dp[amount] > amount ? -1 : dp[amount];
}
// coins=[1,5,11], amount=15 → 3 (11+1+1+1... wait, 5+5+5=3 ✓)
// GREEDY fails: coins=[1,3,4], amount=6 → greedy picks 4+1+1=3, DP gives 3+3=2
```

■ **Time: O(amount * coins) | ■ Space: O(amount)**

Q13. Longest Increasing Subsequence (LIS) — Medium

■ DP · Binary Search · Patience Sort

```
// O(n log n) — Binary Search approach
public int lengthOfLIS(int[] nums) {
List<Integer> tails = new ArrayList<>();
for (int num : nums) {
int pos = Collections.binarySearch(tails, num);
if (pos < 0) pos = -(pos + 1); // insertion point
if (pos == tails.size()) tails.add(num);
else tails.set(pos, num);
}
return tails.size();
}
// [10,9,2,5,3,7,101,18] → 4 (2,3,7,18 or 2,5,7,18)
```

■ **Time: O(n log n) | ■ Space: O(n)**

Q14. 0/1 Knapsack Problem — Medium

■ DP · 2D DP · Space Optimization

```
// Space-optimized 1D DP
public int knapsack(int[] weights, int[] values, int capacity) {
```

```
    int n = weights.length;
    int[] dp = new int[capacity + 1];
    for (int i = 0; i < n; i++) {
    for (int w = capacity; w >= weights[i]; w--) { // iterate BACKWARDS!
    dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
    }
    }
    return dp[capacity];
    }
    // Why backwards? Prevents using same item twice (0/1 constraint)
```

■ Time: O(n * capacity) | ■ Space: O(capacity)

SECTION 5 · GRAPHS, STACK & ADVANCED

Q15. Number of Islands (DFS / BFS on 2D Grid) Medium

■ Graph · DFS · BFS · Matrix

```
public int numIslands(char[][] grid) {
int count = 0;
for (int i = 0; i < grid.length; i++) {
for (int j = 0; j < grid[0].length; j++) {
if (grid[i][j] == '1') {
dfs(grid, i, j);
count++;
}
}
}
return count;
}
private void dfs(char[][] grid, int i, int j) {
if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length
|| grid[i][j] != '1') return;
grid[i][j] = '0'; // mark visited (in-place)
dfs(grid, i+1, j); dfs(grid, i-1, j);
dfs(grid, i, j+1); dfs(grid, i, j-1);
}
```

■ **Time: O(m*n) | ■ Space: O(m*n) DFS stack**

Q16. Topological Sort (Course Schedule — detect cycle + order) Medium

■ Graph · BFS · Kahn's Algorithm

```
// Kahn's BFS topological sort
public boolean canFinish(int numCourses, int[][] prerequisites) {
int[] inDegree = new int[numCourses];
List<List<Integer>> adj = new ArrayList<>();
for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
for (int[] pre : prerequisites) {
adj.get(pre[1]).add(pre[0]);
inDegree[pre[0]]++;
}
Queue<Integer> queue = new LinkedList<>();
for (int i = 0; i < numCourses; i++)
if (inDegree[i] == 0) queue.offer(i);
int processed = 0;
while (!queue.isEmpty()) {
```

```
int course = queue.poll();
processed++;
for (int next : adj.get(course))
if (--inDegree[next] == 0) queue.offer(next);
}
return processed == numCourses; // false = cycle exists
}
```

■ **Time: O(V+E) | ■ Space: O(V+E)**

Q17. Valid Parentheses + Decode String using Stack — Medium

■ Stack · String · Recursion

```
// Valid Parentheses
public boolean isValid(String s) {
Stack<Character> stack = new Stack<>();
for (char c : s.toCharArray()) {
if (c == '(' || c == '[' || c == '{') { stack.push(c); }
else {
if (stack.isEmpty()) return false;
char top = stack.pop();
if (c == ')' && top != '(') return false;
if (c == ']' && top != '[') return false;
if (c == '}' && top != '{') return false;
}
}
return stack.isEmpty();
}

// Decode String: 3[a2[bc]] → abcbcabcbcabcbc
public String decodeString(String s) {
Stack<Integer> counts = new Stack<>();
Stack<String> strings = new Stack<>();
String current = ""; int k = 0;
for (char c : s.toCharArray()) {
if (Character.isDigit(c)) { k = k * 10 + (c - '0'); }
else if (c == '[') { counts.push(k); strings.push(current); k = 0; current = ""; }
else if (c == ']') {
int repeat = counts.pop();
String prev = strings.pop();
current = prev + current.repeat(repeat);
} else { current += c; }
}
return current;
}
```

■ **Time: O(n) | ■ Space: O(n)**

Q18. Word Search in 2D Grid (Backtracking) Medium

■ Backtracking · DFS · Matrix

```
public boolean exist(char[][] board, String word) {
for (int i = 0; i < board.length; i++)
for (int j = 0; j < board[0].length; j++)
if (dfs(board, word, i, j, 0)) return true;
return false;
}

private boolean dfs(char[][] board, String word, int i, int j, int idx) {
if (idx == word.length()) return true;
if (i < 0 || i >= board.length || j < 0 || j >= board[0].length
|| board[i][j] != word.charAt(idx)) return false;
char temp = board[i][j];
board[i][j] = '#'; // mark visited
boolean found = dfs(board, word, i+1, j, idx+1) ||
dfs(board, word, i-1, j, idx+1) ||
dfs(board, word, i, j+1, idx+1) ||
dfs(board, word, i, j-1, idx+1);
board[i][j] = temp; // restore
return found;
}
```

■ **Time: O(m*n*4^L) | ■ Space: O(L) — L = word length**

SECTION 6 · CONCURRENCY CODING (Paytm Favorite)

Q19. Producer-Consumer Problem using BlockingQueue — Medium

■ Concurrency · BlockingQueue · Thread Safety

```
// Paytm: payment event producer + async processor
BlockingQueue<PaymentEvent> queue = new LinkedBlockingQueue<>(1000);

// Producer Thread
class Producer implements Runnable {
public void run() {
while (true) {
PaymentEvent event = generateEvent();
queue.put(event); // blocks if queue full
}
}
}

// Consumer Thread
class Consumer implements Runnable {
public void run() {
while (true) {
PaymentEvent event = queue.take(); // blocks if empty
processPayment(event);
}
}
}

// Launch with thread pool
ExecutorService exec = Executors.newFixedThreadPool(10);
exec.submit(new Producer());
for (int i = 0; i < 5; i++) exec.submit(new Consumer());
```

■ ***Tip:*** *BlockingQueue handles all the locking/signaling. LinkedBlockingQueue is bounded (backpressure). ArrayBlockingQueue is slightly faster but fixed size. PriorityBlockingQueue for priority payments.*

Q20. Implement Bounded Blocking Queue from scratch (no BlockingQueue) — Hard

■ Concurrency · Lock · Condition · Monitor

```
public class BoundedBlockingQueue<T> {
private final Queue<T> queue = new LinkedList<>();
private final int capacity;
private final ReentrantLock lock = new ReentrantLock();
private final Condition notFull = lock.newCondition();
private final Condition notEmpty = lock.newCondition();
```

```
public BoundedBlockingQueue(int capacity) { this.capacity = capacity; }

public void put(T item) throws InterruptedException {
lock.lock();
try {
while (queue.size() == capacity) notFull.await(); // wait if full
queue.offer(item);
notEmpty.signal(); // notify consumer
} finally { lock.unlock(); }
}

public T take() throws InterruptedException {
lock.lock();
try {
while (queue.isEmpty()) notEmpty.await(); // wait if empty
T item = queue.poll();
notFull.signal(); // notify producer
return item;
} finally { lock.unlock(); }
}
}
// Why while loop (not if)? Spurious wakeups — always recheck condition
```

■ **Time: O(1) put/take | ■ Space: O(n)**

■ ***Tip:*** *Classic Senior-level question! Key points: (1) while not if for spurious wakeups (2) two separate Conditions for full/empty (3) always unlock in finally (4) signal not signalAll for performance.*

PATTERNS CHEATSHEET · Recognize & Apply Fast

Pattern	When to Apply	Key Examples
Two Pointers	Sorted array, find pair/triplet, palindrome check	Two Sum (sorted), 3Sum, Container With Most Water
Sliding Window	Subarray/substring max/min with constraint	Longest substring, Max sum subarray of size K
Fast & Slow Pointers	Cycle detection, middle of list, nth from end	Linked list cycle, Floyd's algorithm
Merge Intervals	Overlapping intervals — merge, count, insert	Meeting rooms, calendar conflicts
BFS (Level Order)	Shortest path, level traversal, word ladder	Binary tree levels, shortest path in grid
DFS + Backtracking	All possible combinations, permutations, paths	Word Search, N-Queens, Subsets
Binary Search	Sorted array, answer space is monotonic	Search in rotated, Koko eating bananas
Top-K Elements	K largest/smallest, K most frequent	Kth largest (heap), K closest points
Dynamic Programming	Optimal substructure + overlapping subproblems	Knapsack, Coin Change, LIS, Edit Distance
Monotonic Stack	Next Greater/Smaller element in O(n)	Daily Temperatures, Largest Rectangle Histogram

Time Complexity Quick Reference

Operation	Array	HashMap	BST (balanced)	Heap
Search	O(n)	O(1) avg	O(log n)	O(n)
Insert	O(1) amort	O(1) avg	O(log n)	O(log n)
Delete	O(n)	O(1) avg	O(log n)	O(log n)
Min/Max	O(n)	O(n)	O(log n)	O(1)
Sorted iter	O(n log n)	N/A	O(n)	O(n log n)

Master These 10 Patterns — Solve 80% of Paytm DSA Questions! ■

Practice on LeetCode: Arrays → Linked List → Trees → DP → Graphs → Concurrency